

NTG System

Technical Manual

Architecture, platform, games, data sources, deployment on free tiers and the guide for adding new games.

Version 2.0 · Oct 27, 2026

Angular 21 · Spring Boot 4.0 · PostgreSQL and Storage on Supabase

Contents

1 Introduction
- 1.1 Scope and audience
- 1.2 Key design principles
- 1.3 Game modules at a glance
- 1.4 How this manual is organized

2 Architecture
- 2.1 Components
- 2.2 Request path
- 2.3 Repository layout
- 2.4 How a game plugs in

3 Platform backend
- 3.1 Modules
- 3.2 Package structure
- 3.3 Feature layout
- 3.4 Security
- 3.5 Accounts and session
- 3.6 Game catalog
- 3.7 Public pictures
- 3.8 Errors and logging
- 3.9 Free-tier efficiency

4 Frontend
- 4.1 Technical choices
- 4.2 Folder structure
- 4.3 Routing and access
- 4.4 Session handling
- 4.5 Look and feel
- 4.6 Conventions

5 Database and storage
- 5.1 Schemas and SQL files
- 5.2 Conventions
- 5.3 Schema core
- 5.4 Storage buckets
- 5.5 Working with the schema

6 API reference
- 6.1 Platform endpoints
- 6.2 Conventions

7 Pokémon module
- 7.1 Concepts
- 7.2 Code
- 7.3 Tournament rules
- 7.4 Tables
- 7.5 Endpoints

7.6 Screens
7.7 Team pictures
7.8 Running a tournament
7.9 Data of the old app
7.10 Maintenance cheat sheet

8 WoW Forever module
8.1 Tools
8.2 Where the data comes from
8.3 Code
8.4 How the game data is served
8.5 The data file (data.json)
8.6 The configuration file (config.json)
8.7 The tools
8.8 Pictures and icons
8.9 Updating the game data
8.10 Licence, legal notice and release

9 Setup, build and deployment
9.1 Requirements
9.2 Configuration
9.3 Build and run
9.4 Deployment
9.5 Troubleshooting

10 Adding a new game
10.1 Integration points at a glance
10.2 Backend
10.3 Database
10.4 Frontend
10.5 Release
10.6 Definition of done

11 Appendix
11.1 Conventions
11.2 Glossary
11.3 Known gaps and ideas
11.4 Changelog

1 Introduction

NTG System is a private full-stack web platform where a closed group of friends tracks, organizes and plays everything around the games they share. This manual is the technical reference: how the platform is built, how each game works, where its data comes from and how to keep it up to date, and how to add a new game.

1.1 Scope and audience

Scope. The Spring Boot backend, the Angular frontend, the PostgreSQL data model and the Storage buckets, the security model, deployment on the free plans of Render, Vercel and Supabase, the Pokémon and WoW Forever modules, and the procedure for adding a game. End-user instructions are out of scope.

Audience. Developers and maintainers with working knowledge of Java, Spring, Angular and SQL.

1.2 Key design principles

- **Invitation-only access:** there is no public sign-up; admins create every account.
- **Access by tags:** each game is unlocked by a tag. The backend enforces it; the frontend only hides what a user cannot open.
- **Independent games:** a game is one Maven module and one frontend folder. It only knows a small module of shared contracts, never the platform or another game, and the platform never names a game.
- **The backend owns the data:** catalog, texts, rules and picture URLs come from the API, so the web client only shows them and another client could reuse the same API.
- **Deny-by-default security:** every route must be listed explicitly; anything else is rejected.
- **Built for free tiers:** caches, compressed and conditional responses and a lean deployment keep the site inside the free plans.
- **Every screen works on phones,** from 320 px to desktop.

1.3 Game modules at a glance

Game	Backend	Frontend	What it offers
Pokémon (`pokemon`)	Module `games/pokemon`, schema `pokemon`	`games/pokemon`	Games, randomlockes, tournaments (two groups and a double-elimination bracket, or a league), best-of-3 series, standings, rewards, team pictures, all-time ranking and type calculator. See chapter 7
WoW Forever (`wow-forever`)	Module `games/wow-forever`, no tables	`games/wow-forever`	Talent calculator, spellbook, Legacy System, races and racials, from a data file. See chapter 8

Game	Backend	Frontend	What it offers
League of Legends (`lol`)	None yet	None yet	Catalog entry only, coming soon
Counter Strike 2 (`cs2`)	None yet	None yet	Catalog entry only, coming soon

1.4 How this manual is organized

Chapter	What you will find
2 Architecture	Components, request path, repository layout and how a game plugs into the platform
3 Platform backend	Modules, security, accounts, game catalog, pictures, errors and free-tier techniques
4 Frontend	Structure, routing, session, look and feel, responsive design
5 Database and storage	Schemas, the core tables, Storage buckets and how the SQL files are used
6 API reference	Platform endpoints and the conventions shared by every endpoint
7 Pokémon module	Everything about Pokémon: rules, tables, endpoints, screens, pictures, operation
8 WoW Forever module	Everything about WoW Forever: data files, endpoints, tools, pictures, updating the data
9 Setup, build and deployment	Requirements, configuration, build, hosting and troubleshooting
10 Adding a new game	Step-by-step procedure and definition of done
11 Appendix	Conventions, glossary, known gaps and changelog

Each game is described once, in its own chapter: its backend, tables, endpoints, screens and maintenance live together.

2 Architecture

The browser only talks to the website's own domain: Vercel serves the Angular application and forwards every `/api/*` call to the Spring Boot API on Render, which uses Supabase for identity, data and files.

2.1 Components

Component	Technology	Hosting	Responsibility
Frontend	Angular 21, TypeScript 5.9, RxJS, SCSS	Vercel	Screens, routing, session handling, browser security headers
Backend	Java 25, Spring Boot 4.0, Spring Security, Spring Data JPA	Render (Docker)	REST API, authorization, business rules, picture compression
Identity	Supabase Auth	Supabase	Credentials, emails, JWT access tokens, rotating refresh tokens
Database	PostgreSQL	Supabase	Application data: schema `core` plus one schema per game that needs tables
Files	Supabase Storage	Supabase	Public pictures (covers, game artwork) and private ones (Pokémon teams)

2.2 Request path

1. The browser requests a page or an `/api/v1/...` URL on the site's domain.
2. Vercel returns `index.html` for client routes and rewrites `/api/*` to the backend (`frontend/vercel.json`).
3. The backend assigns a request id, applies CORS and verifies the bearer token locally.
4. Role and tags come from a short in-memory cache, and the deny-by-default rules decide the outcome.
5. Controller, service and repository run the use case against PostgreSQL (or a data file, for static game data).

> **WHY THE REWRITE MATTERS** Because the API is reached through the site's own domain, the refresh-token cookie is first-party, works in browsers that block third-party cookies, and the backend address is never exposed to the web app. Public pictures are the only files the browser downloads from another host (Supabase Storage).

2.3 Repository layout

```
NTG/
├── README.md · LICENSE
├── docs/                          This technical manual
├── backend/                       Spring Boot API · Maven multi-module
│   ├── pom.xml · Dockerfile · .mvn/jvm.config
│   ├── platform-api/              Contracts shared by core and the games
│   ├── core/                      The platform (+ sql/core.sql)
│   ├── games/pokemon/             Pokémon (+ sql/pokemon.sql)
│   ├── games/wow-forever/         WoW Forever (data.json, config.json)
│   └── app/                       Application.java, application.yml → ntg.jar
└── frontend/                      Angular single-page application
    ├── public/                    favicon.png
    ├── proxy.conf.json            Development: /api → localhost:8080
    ├── vercel.json                Rewrite, SPA fallback, cache and security headers
    └── src/app/                   core · shared · layout · features · games
```

2.4 How a game plugs in

A game declares itself on both sides, and the platform builds everything else from that declaration:

Side	Declaration	What the platform does with it
Backend	`GameModule` bean: tag key, API path and the write routes it opens to players	Builds the security rules of the game: read with the tag (or admin), write as admin, plus the declared player writes
Backend	`ImageSlotProvider` bean (optional): the public pictures it needs	Lists them in the admin Pictures screen and accepts uploads only for those keys
Frontend	`GameModule` in `<game>.game.ts`: id, lazy routes, admin routes and admin menu	Mounts `/<id>` and `/admin/<id>` and adds the admin links to the sidebar
Database	A row in the catalog (`core.game`), created from the admin Games screen	Name, order, cover and coming-soon state; creating it also creates the tag

The id of a game is the same everywhere: tag key, API path segment, frontend folder, URL and catalog key.

3 Platform backend

The backend is split into Maven modules so that the platform and the games only meet through a small module of contracts.

3.1 Modules

Module	Depends on	Role
`platform-api`	spring-web	Contracts shared by everyone: `CurrentUser`, `ObjectStorage`, `GameModule`, `ImageCompressor`, `PublicAssets`, `ImageSlotProvider`, `ApiErrors`, `JsonResponses`
`core`	platform-api	Users, tags, game catalog, public pictures, security, Supabase Auth and Storage clients, picture compression, logging and error handling. Implements the contracts
`games/<game>`	platform-api only	Domain and REST endpoints of one game. No security code and no dependency on core
`app`	core and every game	`Application.java` in the root package `com.adrianpmdev`, `application.yml`, and the runnable `ntg.jar`

Because `Application` lives in the root package, Spring finds the components, entities and repositories of every module on the classpath without listing them. The parent POM holds Java 25, the Spring Boot version, the versions of the internal modules and of the libraries (springdoc, Lombok, Bucket4j), the compiler settings, UTF-8 encoding and the test runner.

3.2 Package structure

```
com.adrianpmdev.platform              (platform-api)
├── port/            CurrentUser, ObjectStorage
├── module/          GameModule
├── image/           ImageCompressor, ImageSpec, CompressedImage
├── asset/           ImageSlot, ImageSlotProvider, PublicAssets
├── error/           ApiErrors (404, 409, 400, 403)
└── web/             JsonResponses (JSON with ETag and cache rules)

com.adrianpmdev.core                  (core)
├── config/          AppProperties, OpenAPI, HTTP client, UploadConfig
├── security/
│   ├── config/          Security chains, CORS, JWT decoder
│   ├── authentication/  AuthenticatedUser, CurrentUser impl, UserAuthorityCache,
Swagger login
│   ├── authorization/   Roles and tag authorities
│   ├── ratelimit/       AuthRateLimiter, TooManyRequestsException
│   ├── cookie/          Refresh-token cookie
│   └── filter/          Bearer-token filter
├── image/           JdkImageCompressor
├── integration/supabase/   Auth and Storage clients, DTOs, settings
├── web/             Keep-alive, global exception handler
├── logging/         Request id and access log
└── features/        auth · users · tags · catalog · assets
```

3.3 Feature layout

Every feature, in core and in every game, follows the same layers:

```
<feature>/
├── api/             controller · dto/request · dto/response · mapper
├── application/service/       Business rules and orchestration
├── domain/          Pure rules without Spring (only where needed)
└── infrastructure/persistence/  entity · repository
```

- Controllers never touch repositories; requests are validated at the edge; responses are DTOs, so entities never reach the API.
- A feature that only serves files (the WoW game data, the Pokémon type chart) is just a controller and a service.
- `domain/` holds pure classes with unit tests, as the Pokémon tournament rules do.

3.4 Security

3.4.1 Filter chains

- **Swagger chain (order 1)** covers `/swagger-ui/**` and `/v3/api-docs/**` with HTTP Basic, admin only, checked against Supabase. Swagger is off unless an environment turns it on.
- **API chain (order 2)** is stateless, with CSRF, form login and HTTP Basic disabled. `SupabaseJwtAuthFilter` runs before the standard authentication filter.

3.4.2 Authentication

1. Read the `Authorization: Bearer` header.
2. Verify signature, expiry, issuer and audience locally (`JwtDecoderConfig`): ES256 through the project's JWKS URL by default, or the legacy HS256 secret when `supabase.jwt.secret` is set. The audience must be `authenticated` (`supabase.jwt.audience`), so only session tokens are accepted.
3. Get role and tags from `UserAuthorityCache`: kept 60 seconds, evicted (again after commit) when a user's tags change, a user is created or deleted, or a tag is deleted.
4. Grant `ROLE_<ROLE>` and one `TAG_<KEY>` per tag, except the tags of games that are coming soon in the catalog, which non-admins do not get. A valid token without a local profile stays unauthenticated.

3.4.3 Authorization rules

Deny by default (`anyRequest().denyAll()`):

Route	Method	Access
`/error`, `/keep-alive`, `OPTIONS /**`	any	Public
`/api/v1/auth/login`, `/refresh`, `/logout`	POST	Public, with attempt limits
`/api/v1/auth/set-password`	PUT	Public (needs the link token), with attempt limits

Route	Method	Access
`/api/v1/auth/me`	GET	Signed in
`/api/v1/users/me`	PUT	Signed in (own profile)
`/api/v1/users/**`, `/api/v1/tags/**`, `/api/v1/assets/**`	any	Admin
`/api/v1/games`	GET	Signed in
`/api/v1/games/**`	other	Admin
Player writes declared by a `GameModule`	declared	Admin or the game's tag; the game then checks the caller (for example, the owner of a team picture)
`/api/v1/<game>/**`	GET	Admin or the game's tag
`/api/v1/<game>/**`	other	Admin
Anything else	any	Denied

The game rules are built from the `GameModule` beans on the classpath, so `SecurityConfig` never names a game.

3.4.4 Sign-in limits

The backend signs in against Supabase from its own address, so Supabase's per-address limits cannot protect each user. `AuthRateLimiter` keeps token buckets (Bucket4j) in memory:

Endpoint	Limit
`POST /auth/login`	10 per email every 15 minutes, 20 per client address every 15 minutes, 100 per minute site-wide
`POST /auth/refresh`	30 per client address per minute (only when a refresh cookie is sent)
`PUT /auth/set-password`	5 per client address every 15 minutes, 20 per minute site-wide

A refused attempt answers `429` with `Retry-After`. The client address is the first entry of `X-Forwarded-For`; whoever calls Render directly can fake it, which is why the per-email and site-wide limits exist.

3.4.5 Other protections

Concern	Handling
Passwords	Never stored locally; delegated to Supabase Auth. Policy: length, at least a letter and a number
Sessions	Access token in memory on the client; refresh token in an HttpOnly cookie scoped to `/api/v1/auth`, rotated on every use and revoked on logout

Concern	Handling
Accounts	No public sign-up. Admins create users; users choose a password from a one-time email link. An admin cannot delete their own account
Uploads	Pictures validated by their bytes and re-encoded by the server (metadata dropped); 4 MB per request
Database	Row level security on every table with no policies, so the Supabase Data API reads nothing; the backend connects as the owner
Browser	Content Security Policy and security headers sent by Vercel (see Look and feel and chapter 9)
CORS	Explicit origin allow-list (`app.cors.allowed-origins`), with credentials
Errors	RFC 7807 problem details; server errors return a generic message and a request id, never stack traces
Container	The Docker image runs as an unprivileged user
Secrets	Outside the repository: environment variables or an ignored local file
Audit	Admin actions are written to a dedicated `AUDIT` log

3.5 Accounts and session

- **Creating a user.** `POST /users` with email, username and tags creates a confirmed Supabase account without a password, saves the profile and tags, and asks Supabase to send the "choose your password" email. If saving the profile fails the Supabase account is removed; if only the email fails the user is kept and the email can be resent with `POST /users/{id}/invite`.
- **First access.** The link opens `/set-password`; the frontend calls `PUT /auth/set-password` with the link's token and the backend sets the password and returns a session.
- **Staying signed in.** `POST /auth/login` returns an access token and sets the refresh cookie; `POST /auth/refresh` reads only the cookie, rotates it and returns a new access token.

3.6 Game catalog

`core.game` lists the games the web clients show. `GET /games` answers each user with the games they can see: admins get every game; players get the released games they hold the tag for, plus the coming-soon ones (listed, not accessible). Each entry carries `id`, `name`, `coverUrl`, `upcoming`, `accessible` and `sortOrder`.

- **Creating a game** (admin Games screen or `POST /games`) also creates its tag when it does not exist, in the same transaction. New games start as coming soon.
- **Releasing a game** is unticking coming soon: players with the tag can open it from their next request. No deploy is needed.
- **Removing a game** deletes its cover but keeps its tag and its data.

- The catalog is kept in memory and reloaded after every change, because the token filter reads it on every request.

3.7 Public pictures

Covers and game artwork are pictures that admins upload from the Pictures screen. Modules declare which ones they need (`ImageSlotProvider`), each with a key such as `platform/games/pokemon` or `wow-forever/talents/warrior-arms`; uploads are only accepted for declared keys.

Step	What happens
Validate	The content must be JPEG or PNG (first bytes, never the name), at most 4 MB and 40 megapixels
Compress	`JdkImageCompressor` (JDK only): resize to the longest side allowed (1,920 px for public pictures), re-encode as JPEG, or PNG when there are transparent pixels, and lower the quality (0.82, 0.72, 0.62) and then the size until the file fits (1.5 MB). One picture at a time, to protect the memory of the free server
Store	Public bucket `public-assets`, object `<key>.<jpg|png>`, overwritten on every upload; `core.asset` records the object, size and dimensions
Serve	URL `.../object/public/public-assets/<object>?v=<upload time>`: it changes with every upload, so browsers cache each version for a year. URLs are kept in memory and rebuilt after a change

The same compressor serves the Pokémon team pictures (1,600 px, 1 MB, private bucket; see chapter 7).

3.8 Errors and logging

`GlobalExceptionHandler` maps validation errors, malformed input, missing resources, integrity violations (`409`), oversized uploads (`413`), security errors and deliberate business errors to problem details with `title`, `status`, `detail`, `path` and `requestId`.

`RequestLoggingFilter` runs first, accepts a valid `X-Request-Id` or generates one, puts it in the logging context and writes one `ACCESS` line per request (except `/keep-alive`). Admin actions write `AUDIT` lines. Logs go to the console only.

3.9 Free-tier efficiency

Technique	Where	Effect
Role and tags cache (Caffeine, 60 s)	`UserAuthorityCache`	At most one query per user per minute instead of one per request
Catalog and picture URLs in memory	`GameCatalogService`, `AssetService`	Reloaded only after an admin change
Gzip	`application.yml`	JSON compressed (WoW data from about 1.8 MB to about 200 KB)

Technique	Where	Effect
ETag on static JSON	`JsonResponses`	A browser that has the data gets `304` with no body
Daily database ping	`KeepAliveController`	The external cron keeps Render awake every 10 minutes; once a day the call also runs `select 1` so Supabase never pauses
Small connection pool	`application.yml`	At most 5 connections to the Supabase pooler
JVM tuned for 512 MB	`Dockerfile`	70% heap, SerialGC, exit on out-of-memory so Render restarts it
Session caches in the client	Frontend services	Catalog and game data requested once per session; Pokémon reads kept one minute
Long browser cache	`vercel.json`	Hashed JS and CSS cached for a year, images for a week

4 Frontend

The frontend is an Angular 21 single-page application in strict TypeScript. Its only runtime dependencies are Angular and RxJS: no UI library and no state library. It shows what the API sends; the data itself (catalog, texts, rules, picture URLs) belongs to the backend.

4.1 Technical choices

- Standalone components, zoneless change detection, signals, lazy-loaded routes and `OnPush` on every component.
- Strict TypeScript and strict templates; SCSS with design tokens exposed as CSS custom properties.
- One API address everywhere: the relative `/api/v1`. `ng serve` forwards it to `localhost:8080` (`proxy.conf.json`) and Vercel forwards it to Render, so local and production behave the same with the session cookie.
- The interface text is in Spanish; code and documentation are in English.

4.2 Folder structure

```
frontend/src/
├── main.ts · index.html · styles.scss
├── styles/                Tokens, mixins (breakpoints), reset, typography, terminal,
modal,
│                          admin-page, auth-screen
└── app/
    ├── app.config.ts      Zoneless change detection, router, HTTP client +
interceptor
    ├── app.routes.ts      Platform routes + the routes of every game (built from
games.ts)
    ├── core/
    │   ├── config/        api.ts (API_BASE_URL), game-module.ts (GameModule contract)
    │   ├── services/      auth, game-catalog, asset, user, tag, notification
    │   ├── interceptors/  auth (token + silent renewal)
    │   └── guards/        auth · admin · role-home · game
    ├── shared/            models (user, tag, game, asset), notifications, utils
    ├── layout/            Terminal shell: main layout, header, footer
    ├── features/          auth · welcome · admin (dashboard, users, games, images)
    └── games/
        ├── games.ts       GAME_MODULES: the games built into this client
        ├── pokemon/       See chapter 7
        └── wow-forever/   See chapter 8
```

Path aliases: `@core`, `@shared`, `@layout`, `@features`, `@games`. Each game keeps its own SCSS partial in `games/<game>/styles/`.

4.3 Routing and access

Route	Who can enter	Layout
`/login`, `/set-password`	Public	Full-screen terminal

Route	Who can enter	Layout
`/`	Signed in	Redirects: admin to the dashboard, player to welcome
`/welcome`	Signed in	Terminal shell
`/admin/dashboard`, `/users`, `/games`, `/images`	Admin	Terminal shell
`/admin/<game>/**`	Admin	Terminal shell (code in the game folder)
`/<game>/**`	Admin, or players the catalog marks as accessible	The game's own layout

`app.routes.ts` builds `/<id>` and `/admin/<id>` for every entry of `GAME_MODULES`. `gameGuard(id)` restores the session itself (guards of one route run in parallel) and asks `GameCatalogService` whether the game is accessible. The catalog is requested once per session, refreshed on the welcome page and after admin changes, and forgotten when the user changes. The sidebar shows the accessible games; a coming-soon game without a module in this client is dimmed and does not unfold.

4.4 Session handling

- The access token lives only in memory; the long session is the HttpOnly refresh cookie, which scripts cannot read.
- On reload, guards call `POST /auth/refresh` to restore the session before showing a page.
- `AuthService` renews the token 60 seconds before it expires (minimum delay 5 seconds); concurrent renewals share one request, because the refresh token rotates.
- `authInterceptor` adds the bearer token. On a `401` it renews once and repeats the request; `/auth/` calls are never retried, so a failed refresh cannot loop.

4.5 Look and feel

Platform areas (login, welcome, admin) share a terminal look: monospaced type, neon accents, command-line flavour. Each game defines its own identity inside its folder; terminal styles are scoped and never leak into a game. Components written against the terminal tokens take a game's palette when the game layout re-points those tokens.

4.5.1 Responsive design

Breakpoints are shared mixins in `styles/_mixins.scss`: `tablet` (900 px), `mobile` (640 px) and `small` (420 px). Every screen was checked at 320, 360, 768 and 1280 px. The patterns used:

Pattern	Where
Rows become cards with labelled lines and the actions below (`data-label` on each cell)	Every `admin-table` and the users table

Pattern	Where
First columns fixed while the rest scrolls sideways	Pokémon standings tables
Scroll snapping: each swipe moves a whole round and the next one peeks in	Pokémon bracket
Tooltips centred on screen when the finger leaves no room on either side	WoW talent and perk tooltips
Trees one per screen with tabs, swiped as a carousel	WoW talent calculator and Legacy
`100dvh` and no automatic text enlarging	Layouts and the base styles

4.5.2 Browser security headers

`vercel.json` sends on every path: a Content Security Policy (`default-src 'self'`; scripts only from the site; styles from the site and Google Fonts; fonts from Google Fonts; images from the site and any HTTPS host; API calls only to the site; no plugins; no framing), HSTS, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, a strict referrer policy, a permissions policy that disables camera, microphone, location, payment and USB, and `Cross-Origin-Opener-Policy`.

> **NO INLINE SCRIPT** The policy forbids inline code. `angular.json` turns off critical-CSS inlining because it adds an inline `onload` handler; keep it off, and never add inline scripts or event attributes to `index.html`.

4.6 Conventions

- Features and games may use `core`, `shared` and `layout`; `core` and `shared` never import from `features` or `games`; a game never imports another game or `features`.
- A file used by one area stays inside it and moves to `core` or `shared` only when two areas need it.
- Hyphen-separated file names, one base name per component (`.ts`, `.html`, `.scss`), no type suffixes, selector prefix `app-`; every file starts with a short English header comment.

5 Database and storage

PostgreSQL on Supabase, with one schema for the platform (`core`) and one for each game that needs tables. Hibernate runs with `ddl-auto=none` and `open-in-view=false`: the schema is created and changed with SQL files kept next to the code of each module.

5.1 Schemas and SQL files

Schema	SQL file	Content
`core`	`backend/core/sql/core.sql`	Users, tags, game catalog, public pictures; the public bucket and its policy
`pokemon`	`backend/games/pokemon/sql/pokemon.sql`	Nine tables of the Pokémon module; the private bucket of team pictures and its policy (see chapter 7)
none	—	WoW Forever keeps its data in files inside its module

On an empty Supabase project run `core.sql` and then each game's file in the SQL editor. The files use `if not exists`, `on conflict` and `drop policy if exists`, so a section can be run again when it changes.

5.2 Conventions

- UUID primary keys, except tables keyed by a text key (`core.tag`, `core.game`, `core.asset`) and join tables.
- snake_case names; timestamps with time zone (`timestamptz`) in every table except `core.user`; statuses as short text values with converters in Java.
- Integrity in the database (unique keys, foreign keys, not null, checks); the exception handler turns violations into clean `409` answers.
- Row level security enabled on every table with no policies: the Supabase Data API (anon and authenticated roles) can read nothing, and the backend connects as the owner.
- A game schema never references `core`: a link to a platform user is a plain UUID column.

> **NAMING EXCEPTION** An id with a hyphen (`wow-forever`) is not a valid unquoted SQL identifier or Java package. The schema would use an underscore (`wow_forever`) and the package drops it (`wowforever`); the tag, API path, catalog key and frontend folder keep the hyphen.

5.3 Schema core

Table	Columns	Notes
`"user"` (quoted: reserved word)	`id`, `email`, `username`, `avatar_url`, `role`, `created_at`	`id` is the Supabase Auth id; `email` copied so users can be listed without Supabase; role `usuario` or `admin`
`tag`	`key`, `name`, `description`, `created_at`	Key immutable (2 to 30 lowercase letters, digits, -, _); becomes the authority `TAG_<KEY>`
`user_tag`	`user_id`, `tag_key`, `assigned_at`	Join table
`game`	`key`, `name`, `upcoming`, `sort_order`, `created_at`	The catalog; key = tag key = game id. Seeded with Pokémon, WoW Forever, LoL and CS2
`asset`	`key`, `object_name`, `content_type`, `size_bytes`, `width`, `height`, `updated_at`	One row per uploaded public picture; the key matches a declared slot

5.4 Storage buckets

Bucket	Visibility	Limits	Content
`public-assets`	Public read by URL; writes only by the backend	2 MB, JPEG or PNG	Covers and game artwork, one object per key
`pokemon-teams`	Private; only the backend with signed URLs	1 MB, WebP, JPEG or PNG	Team pictures of tournament players

A restrictive policy on `storage.objects` denies the anon and authenticated roles every operation on each bucket, so a permissive policy added later by mistake cannot open them. The backend uses the service role key. If creating a policy fails with "must be owner of table objects", the comment in the SQL file explains how to create it from the Storage dashboard.

5.5 Working with the schema

- **Source of truth.** The SQL files and the live database. Update the file of the module with every structural change and run the new part in Supabase before deploying the code that needs it.
- **Credentials.** `spring.datasource.*`, supplied through environment variables or the ignored local properties file.

6 API reference

Every route lives under `/api/v1`, except `GET /keep-alive`. Swagger UI (`/swagger-ui.html`) is available only where an environment enables it, behind admin Basic Auth. The endpoints of each game are listed in its chapter.

6.1 Platform endpoints

Resource	Endpoints	Access
Auth	`POST /auth/login`, `POST /auth/refresh`, `POST /auth/logout`, `PUT /auth/set-password`	Public, with attempt limits
Auth	`GET /auth/me`	Signed in
Profile	`PUT /users/me`	Signed in (own profile; no screen uses it yet)
Users	`POST /users`, `GET /users`, `GET /users/{id}`, `PUT /users/{id}`, `DELETE /users/{id}`, `POST /users/{id}/invite`	Admin
User tags	`PUT /users/{id}/tags`, `POST /users/{id}/tags/{key}`, `DELETE /users/{id}/tags/{key}`	Admin
Tags	`GET /tags`, `POST /tags`, `PUT /tags/{key}`, `DELETE /tags/{key}`	Admin
Catalog	`GET /games`	Signed in (answer depends on the caller)
Catalog	`POST /games`, `PUT /games/{key}`, `DELETE /games/{key}`	Admin
Pictures	`GET /assets`, `PUT /assets?key=` (multipart `file`), `DELETE /assets?key=`	Admin
Health	`GET /keep-alive`	Public

The key of a picture goes as a query parameter because it contains slashes. `GET /assets` lists every declared slot (uploaded or not) grouped as the modules declare them, followed by uploaded pictures no module expects any more.

6.2 Conventions

- Request bodies are validated DTOs and responses are explicit DTOs; entities are never returned.
- `401` means no valid session, `403` no permission, `409` a business or integrity conflict, `413` an upload over 4 MB, `415` a file that is not a supported picture, `429` too many attempts (with `Retry-After`), `502` a Storage failure, `503` a static data file that cannot be read.
- Static JSON (game data, type chart) carries an `ETag` and `Cache-Control`; send `If-None-Match` to get `304` when it did not change.

- Send `X-Request-Id` (8 to 64 letters, digits or hyphens) to correlate a call with the logs; the server returns the id it used.
- Errors use RFC 7807 problem details:

```
{
  "title": "Validation failed",
  "status": 400,
  "detail": "username: Username must have between 3 and 50 characters",
  "path": "/api/v1/users",
  "requestId": "a1b2c3d4e5f6"
}
```

7 Pokémon module

A group of friends plays randomlockes: a run of a Pokémon game with shared rules, a fixed roster and tournaments at fixed points. Each tournament ends with rewards that players pick in the order of its final standings. The area keeps the whole history and lets admins run new tournaments; players only read, except for uploading their own team picture.

7.1 Concepts

Concept	Meaning
Game	A Pokémon game (FireRed, HeartGold...). Tournaments are played in a game
Randomlocke	Rules, a fixed roster and a series of tournaments. It can move from game to game (GenLocke 2025 went from FireRed to HeartGold and Platino)
Tournament	Two groups and a double-elimination bracket, or a league. Every series is best of 3
Standings	Per tournament (decides the reward order), per randomlocke (cumulative, from 0) and all-time
Rewards	Listed per tournament; an admin records who picked each one

7.2 Code

```
backend/games/pokemon/  →  com.adrianpmdev.pokemon
├── PokemonModule.java      GameModule: tag pokemon, /api/v1/pokemon, team pictures
open to players
├── shared/dto/             RefResponse (id + name)
└── features/
    ├── games/ · players/ · lockes/      CRUD, linked accounts, rules (jsonb) and
roster
    ├── tournaments/
    │   ├── api/            Tournament, TournamentPlayer, Match and Reward controllers
+ DTOs
    │   ├── application/    Query, calendar and bracket, series, rewards, team
pictures, results
    │   ├── domain/         RoundRobin, GroupsToBracket, DoubleEliminationBracket,
Standings,
    │   │                   SeriesScore, enums  (unit tests: TournamentRulesTest)
    │   └── infrastructure/ Entities, repositories, enum converters
    ├── rankings/           Randomlocke standings and the all-time ranking
    └── types/              Type chart served from resources/pokemon/types.json

frontend/src/app/games/pokemon/
├── pokemon.game.ts · pokemon.routes.ts · pokemon-admin.routes.ts
├── pokemon-layout/         Header, menu (Juegos, Ranking, Calculadora), theme tokens
├── data/                   PokemonApi, models, Spanish labels, rules document, view
helpers
├── ui/                     Standings table, series card, bracket, calendar, rules,
team gallery,
│                           series editor
├── games/ · game/ · locke/ · tournament/ · ranking/ · type-calculator/
├── admin/                  games · players · lockes · locke · tournament
└── styles/_pokemon.scss
```

The module uses two platform contracts: `CurrentUser` (who is calling and whether they are an admin) and `ObjectStorage` (team pictures), plus `ImageCompressor` for the pictures.

7.3 Tournament rules

- **Formats.** `groups_bracket`: two groups of the same size play a round robin, then everyone goes to a double-elimination bracket. `round_robin`: one league, everybody plays everybody once.
- **Series.** Best of 3: decided when a side wins two games; no game may follow.
- **Points.** Series won × points per win of the tournament, plus the player's `points_adjustment` (imported history, bonuses, penalties).
- **Table order.** Points, game difference, head-to-head among the tied players (the direct series for two, the wins among them for more), then the name.
- **Groups.** With an odd number of players one random player skips the groups and goes straight to the bracket. Groups can also be set by hand.
- **Groups to bracket.** First of one group against last of the other, second against second-to-last, and so on; the bracket is the smallest power of two (4 to 32) that fits.
- **Double elimination.** Upper rounds, lower rounds and one grand final without reset. A series against an empty side is a bye; with two empty sides it is void.
- **Final order.** With a bracket: champion, finalist, then by how late each player fell in the lower side. In a league: the table. It is the order in which players pick rewards.

- **Randomlocke standings** add the points of its tournaments from 0; the all-time ranking adds every randomlocke.

`DoubleEliminationBracket` knows where both sides of every series come from. After each result `ScheduleService.syncBracket` writes who plays each later series; if a corrected result changes the players of a series, its stored games no longer apply and are removed, which can cascade.

> **IMPORTED BRACKETS** The six-player brackets of the old app do not follow that structure. Their tournaments have `bracket_size` null: they are shown as played and nothing advances on its own.

7.4 Tables

Schema `pokemon`, nine tables, all with a UUID `id` except `locke_players`. `players.user_id` holds a platform user id as a plain column, so the schema stays independent from `core`.

Table	Main columns	Rules
`games`	name, description, image_url, created_at	A game with tournaments cannot be deleted
`lockes`	name, description, rules (jsonb), created_at	A randomlocke can span several games
`players`	name, user_id, created_at	Unique name; one player per account
`locke_players`	locke_id, player_id	Fixed roster; new tournaments start with it
`tournaments`	locke_id, game_id, position, name, format, status, points_per_win, teams_reveal_at, bracket_size	Unique (locke_id, position); deleting it cascades
`tournament_players`	tournament_id, player_id, group_name, team_image, points_adjustment	Unique (tournament_id, player_id)
`matches`	tournament_id, stage, round, slot, player1_id, player2_id	Unique (tournament_id, stage, round, slot)
`match_games`	match_id, game_number, winner_side, replay_url, played_at	Game 1 to 3, winner side 1 or 2
`rewards`	tournament_id, sort_order, name, chosen_by	A player picks at most one reward

Column	Allowed values
`tournaments.format`	`groups_bracket`, `round_robin`
`tournaments.status`	`draft`, `in_progress`, `finished` (set by the admin)
`tournaments.bracket_size`	4, 8, 16, 32, or null (no bracket yet, or imported)
`matches.stage`	`group`, `league`, `upper`, `lower`, `grand_final`
`tournament_players.group_name`	`A`, `B` or null

7.5 Endpoints

Under `/api/v1/pokemon`. Reading needs the `pokemon` tag (or admin); every other method needs admin, except the team pictures.

Resource	Endpoints	Notes
Games	`GET /games`, `GET /games/{id}`, `POST`, `PUT /games/{id}`, `DELETE /games/{id}`	Each game includes `lockeCount`
Players	`GET /players`, `POST`, `PUT /players/{id}`, `DELETE /players/{id}`	`userId` only sent to admins
Randomlockes	`GET /lockes[?gameId=]`, `GET /lockes/{id}`, `POST`, `PUT /lockes/{id}`, `PUT /lockes/{id}/players`, `DELETE`	The detail includes tournaments with champion and the standings
Tournaments	`GET /tournaments/{id}`, `POST`, `PUT /tournaments/{id}`, `DELETE`	The detail is the whole page
Tournament actions	`POST /tournaments/{id}/players`, `/schedule`, `/bracket`, `/matches`, `/rewards`	Add a player, calendar (`{ randomGroups }`), bracket, series, reward
Tournament players	`PUT /tournament-players/{id}`, `DELETE /tournament-players/{id}`	Group and points adjustment
Team pictures	`PUT /tournament-players/{id}/team-image` (multipart `file`), `DELETE`	Open to players with the tag; see below
Series	`PUT /matches/{id}/games`, `DELETE /matches/{id}`	Replaces every game of a series (0 to 3)
Rewards	`PUT /rewards/{id}`, `DELETE /rewards/{id}`	`{ name, chosenBy }`
Ranking	`GET /ranking`	All-time ranking
Types	`GET /types`	Types and damage chart; `ETag`, cached one day

Every admin action on a tournament answers with the updated tournament detail. Business errors answer `409` with a message in Spanish that the admin screens show as is (for example "La fase de grupos no ha terminado").

7.6 Screens

Route	Screen
`/pokemon/games`, `/pokemon/games/:gameId`	Games, and the randomlockes of one game
`/pokemon/lockes/:lockeId`	Tournaments with champion, cumulative standings, rules, players

Route	Screen
`/pokemon/tournaments/:tournamentId`	Teams, groups or league, bracket, standings, rewards with pick order
`/pokemon/ranking`	All-time ranking
`/pokemon/type-calculator`	Defense (damage received by one or two types) and attack (best damage of one or two move types). The chart comes from `GET /types`; the calculation runs in the browser
`/admin/pokemon/lockes`, `/players`, `/games`	Management lists
`/admin/pokemon/lockes/:lockeId`, `/tournaments/:tournamentId`	Roster and tournaments; tournament management in tabs (players, calendar, bracket, standings, rewards, settings)

The components in `ui/` are written against the terminal tokens. The Pokémon layout re-points them to its own palette, so the same bracket or table looks yellow and blue for players and green, and editable, in the admin area. `PokemonApi` keeps each read for one minute and empties that memory after any write.

7.7 Team pictures

Concern	Handling
Storage	Private bucket `pokemon-teams`; `tournament_players.team_image` keeps only the object name `tournaments/<tournament>/<entry>-<random>.<ext>`
Who uploads	An admin always; the player linked to that entry while the tournament is not finished and the reveal moment has not passed
Who sees	An admin and the owner always; everybody else once `teams_reveal_at` has passed (or when there is none)
Files	The browser sends the original (JPEG or PNG, up to 4 MB); the platform compressor resizes it to 1,600 px and keeps it under 1 MB
URLs	Signed for one hour, in one batch per tournament, cached until ten minutes before they expire. A Storage failure only leaves the pictures out
Clean-up	Replacing or removing a picture deletes the old object; deleting a tournament or randomlocke deletes its pictures

7.8 Running a tournament

1. **Prepare.** In Gestión › Randomlockes open the randomlocke, check its roster and create the tournament (game, format, points per win, reveal moment). It starts in draft with the roster as players.
2. **Teams.** Linked players upload their own picture until the reveal moment; admins can upload any.
3. **Calendar.** In Grupos or Liga, generate the calendar (optionally drawing the groups) and set the status to in progress.

4. **Results.** Click a series and record each game (winner and replay link). The series closes at two wins.
5. **Bracket.** When every group series is decided, generate the bracket. Results move players forward on their own.
6. **Rewards.** Add the rewards, record who picked each one following the final standings, and finish the tournament.

7.9 Data of the old app

GenLocke 2025 (6 tournaments, 72 series, 165 games, 36 rewards, 31 team pictures) came from a previous app and was migrated once, on 2026-10-27, with one-off scripts removed afterwards; no copy of that data is kept in the public repository. The three GenLocke modalities became one randomlocke; old rows were grouped into series; `points_adjustment` keeps the old placement points so the standings match; the six-player brackets were kept as played.

7.10 Maintenance cheat sheet

When...	Do this
A new player joins	Create the player, link their account, add them to the roster
A player cannot upload their team	Link their account in Gestión › Jugadores; check the reveal moment and that the tournament is open
A result was wrong	Edit the series; later bracket series that change players lose their games
The groups were wrong	Clear the recorded games, fix the groups and generate the calendar again
Pictures do not show	Check the bucket `pokemon-teams` and `supabase.service-role-key`; the log says when Storage fails
A new Pokémon game is played	Add it in Gestión › Juegos and create the next tournament in it
The type chart changes	Edit `resources/pokemon/types.json` and redeploy the backend

8 WoW Forever module

WoW Forever is a set of companion tools for World of Warcraft: Forever, built as an unofficial fan project, with a launcher-style home from which each tool opens. It has no database tables and no management area: its data lives in two files inside the backend module, and its pictures are managed in the platform Pictures screen.

8.1 Tools

Tool	Route	Status
Calculadora de talentos	`/wow-forever/talents`	Plan a build from level 10 to 60 and share it by link
Spellbook	`/wow-forever/spellbook`	Every trainer spell of each class, rank by rank
Legacy System	`/wow-forever/legacy`	Perk planner and challenge checklist
Razas y racials	`/wow-forever/races`	Racials and race-class combinations
Dungeons y loot	`/wow-forever/dungeons`	Not built; shown as "Próximamente". No data source chosen yet

8.2 Where the data comes from

Data	Stored in	How to change it
Talents, spellbooks and spell texts, racials, class highlights, Legacy perks and challenges, classes and races	`data.json` in the backend module	Replace or edit the file and redeploy the backend (Updating the game data)
Texts in Spanish, faction names, tool list, launcher news, talent rules, tree ids of share codes, icon host	`config.json` in the backend module	Edit the file and redeploy the backend
Logo, faction backgrounds, one background per talent tree	Public bucket, uploaded from the Pictures screen	Upload a new picture; no deploy
Every icon (classes, races, talents, spells, perks)	Not stored: Blizzard's icon host	Change the icon name in `data.json` or the host in `config.json`
Ticked Legacy challenges	Each player's browser (`localStorage`)	Nothing to maintain

8.3 Code

```
backend/games/wow-forever/  →  com.adrianpmdev.wowforever
├── WowForeverModule.java     GameModule: tag wow-forever, /api/v1/wow-forever (read
only)
├── features/gamedata/
│   ├── api/controller/       GameDataController
│   ├── api/dto/response/     GameDataResponses (the API models)
│   └── application/service/  GameDataService, GameDataMapper, GameDataConfig,
GameImagesService
└── resources/wow-forever/    data.json · config.json

frontend/src/app/games/wow-forever/
├── wow-forever.game.ts · wow-forever.routes.ts
├── wow-forever-config.ts     Credit line and broken-icon helper
├── game-data/                GameDataApi (session cache), models, game-images
(picture URLs)
├── home/                     Launcher: home, launcher-header, news-panel
├── talents/                  Calculator, tree grid, tooltip, pure rules
(talents-logic)
├── spellbook/ · legacy/ · racials/
├── ui/                       wow-page, class-picker, wow-button, wow-frame,
legal-notice
└── styles/_wow-forever.scss  Palette, panels, tabs, three-tree layout (carousel on
phones)
```

8.4 How the game data is served

`GameDataService` reads `data.json` and `config.json` the first time an endpoint is called, checks that `data.json` has the sections `talents`, `spellbooks`, `spell_desc`, `racials`, `class_racials`, `class_abilities` and `legacy`, and keeps both in memory. `GameDataMapper` turns the sections into the API models (`GameDataResponses`): prerequisites resolved inside each tree, the game's own tree ids, Spanish texts from `config.json` and full icon URLs. Each response is serialized once and kept.

Endpoint	Answer
`GET /data`	Credit, classes, tools and news of the home, faction names, the levels that give a talent point
`GET /data/talents/{classSlug}`	The trees of one class
`GET /data/spellbook/{classSlug}`	Tabs (General first), spells with every rank, highlights and removed spells
`GET /data/racials`	Races of both factions, their racials and the class abilities that change with the race
`GET /data/legacy`	Perk trees, challenges grouped with Spanish names, points
`GET /data/images`	URLs of the uploaded pictures by short key (`logo`, `backgrounds/horde`, `talents/warrior-arms`…); not cached

Data responses carry `Cache-Control: max-age=3600, private` and an `ETag`. The class slug is the class name in lowercase with every run of other characters turned into `-` (Warrior gives `warrior`); an unknown slug answers `404`. The frontend asks for each response once per session.

> **IMPORTANT** If `data.json` or `config.json` is missing or invalid, or `data.json` lacks a required section, only the game data endpoints answer `503`; the rest of the backend keeps working and the files are read again on the next request. Missing fields inside a section become empty values. `GameDataServiceTest` checks the responses built from the real files.

8.5 The data file (data.json)

Source and licence. The export that talentsforever.com publishes at `https://talentsforever.com/data.json`, read from the WoW Forever client (the General tab of each spellbook is kept by hand by the site). Licensed **CC BY 4.0**: it may be copied and adapted with credit and a link to talentsforever.com. Game names, icons and tooltip text belong to Blizzard Entertainment. Every tool page shows the credit line.

Section	Content	Used by
`license`, `attribution`, `generated`	Licence, credit line and export date	Credit line
`talents`	Per class: icon, source (with the game build), trees with talents (name, max, row and col from 1, icon, desc per rank, req, cost, classic)	Talent calculator; the class list
`spellbooks`	Per class: general and tabs as [spell, rank], levels, talents, icons, gone, build	Spellbook
`spell_desc`	Tooltip of every rank, keyed `Class|Spell|Rank`	Spellbook
`class_abilities`	Per class: what changed, as [name, text, icon]	Spellbook highlights
`racials`, `class_racials`	Races by faction with classes and abilities; class abilities by race	Races and racials
`legacy`	points, source, trees with perks (gate, req, placeholder), challenges (group, name, desc, points)	Legacy System

> **KEEP THE NAMES** The class list is the keys of `talents`, and every other section finds a class by that name; a race must be written the same in `racials` and `class_racials`.

Current content	Value
Export generated	2026-10-26
Game build	1.60.1.70425 (some classes carry later hotfixes; their source line says so)
Classes / trees / talents	9 classes, 27 trees, 465 talents
Spellbooks	9 spellbooks, 585 spells, 1,826 rank texts
Races	10: five Alliance and five Horde, including the two Skyborne
Legacy	3 trees (ids 1187 to 1189), 16 points per character, 65 challenges
Share codes	24 of the 27 trees match wow-forever.gg exactly; Hunter Marksmanship, Mage Arcane and Mage Frost follow an older build

8.6 The configuration file (config.json)

Field	Meaning
`iconBaseUrl`, `fallbackIcon`	Icon host (`render.worldofwarcraft.com/us/icons/56`) and the question-mark icon for missing ones
`factions`	Spanish names of ALLIANCE and HORDE
`features`	Tools of the home: id (also the route segment), label, description, available
`news`	Launcher news: id, title, optional date and image (today static placeholders)
`talents`	`firstPointLevel` (10) and `lastPointLevel` (60), `pointsPerRow` (5), `treeIds` (game ids by `<class>-<tree>`)
`spellbook`	Name of the General tab and where each rank is learned, by trainer code
`legacy`	Name of placeholder perks and Spanish names of the challenge groups

8.7 The tools

Talent calculator

- One point per level from 10 to 60 (51 at level 60). Row N opens with 5 × (N − 1) points in the rows above; a talent with a prerequisite also needs it at full rank.
- Click adds a point; right click, a long press (450 ms) or Backspace/Delete removes one if the rest of the build stays valid. The level cannot go below what the spent points need.
- The state lives in the URL: `?class=<slug>&level=<n>&build=<code>`. The code has one group per tree with points, `<tree id>.<digits>` (one base-32 digit per talent, top row first, trailing zeros trimmed), groups joined with `-`. A code that does not fit the current trees opens empty.

Spellbook

Class picker, a folding panel with what changed for the class, one tab per spellbook tab, a search that ignores case and accents, and a level limit. Each spell shows its ranks, tooltip lines, school, coefficient, where it is learned, notes and the Classic text.

Legacy System

The three perk trees reuse the talent grid, tooltip and rules; a perk gate counts every other point of its tree instead of the rows above. Placeholder perks show as "Próximamente". Challenges are listed by group with a checkbox each, kept in the browser (`ntg.wow-forever.legacy.completed`).

Races and racials

Faction tabs, one card per race with its classes, racials and the class abilities that change with it, and a race-class table.

8.8 Pictures and icons

Picture key (under wow-forever/)	Used for
`logo`	Logo in the launcher header
`backgrounds/alliance`, `backgrounds/horde`	Launcher background, switched by the faction selector
`talents/<class>-<tree>`	Background of each tree; one slot per tree of `data.json` (27 today)

`GameImagesService` declares these slots, so a new class or tree asks for its picture on its own in the Pictures screen. Icons are not stored: they load from Blizzard's host; a missing image is hidden and the name next to it still identifies the item.

8.9 Updating the game data

Step 1 · Get or edit the file

Download `https://talentsforever.com/data.json` and check that `generated` is newer than in the table above, or edit the current file. Keep the indentation: diffs stay small.

Step 2 · Check it

```
node -e "const d=require('./data.json');console.log(d.generated,
  Object.keys(d.talents).join(' '),
  Object.keys(d.spell_desc).length+' spell texts',
  d.legacy.challenges.length+' challenges')"
```

Step 3 · Replace it

Copy it over `backend/games/wow-forever/src/main/resources/wow-forever/data.json`, same path and name.

Step 4 · Check what else it needs

If the new file...	Then
Only changes numbers, texts, ranks, perks or challenges	Nothing else
Adds or renames a tree or a class	Upload its tree picture in the Pictures screen (the slot appears on its own) and add its game id to `talents.treeIds` in `config.json`; without it the tree uses its position as id
Moves or adds talents inside a tree	Nothing to fix, but share links made before may open a different or empty build
Adds a Legacy challenge group	Optional: add its Spanish name to `legacy.challengeGroups` in `config.json`
Changes its structure	Adapt `GameDataMapper` (and `GameDataResponses` if the models change), and `REQUIRED_SECTIONS` if a required section is renamed

Step 5 · Test and deploy

Run `mvn clean install` (the game data tests use the real file), start the backend, open every tool as an admin and check the credit line shows the new build. Then commit and push: Render rebuilds the image with the new file. Browsers may keep the previous response up to one hour. Update the content table above and the changelog. To go back, restore the previous file from Git and redeploy.

8.9.1 If something goes wrong

Symptom	Likely cause
The data endpoints answer 503	A file is missing or invalid, or a required section is missing; the log says which
A tool shows "No se han podido cargar los datos"	Its request failed; the Network tab shows which one
A class is missing, or opens without spellbook or racials	It is not a key of `talents`, or its name differs between sections
A tree has no background	Its picture has not been uploaded
An icon shows a question mark or nothing	The file gives no icon, or the name has no image on the host

8.10 Licence, legal notice and release

- Keep the credit line on every tool page: CC BY 4.0 requires attribution with a link to talentsforever.com. `LICENSE` records the licence of the file.
- The launcher footer says the project is unofficial and not affiliated with Blizzard Entertainment. Do not remove it.
- To open the module to players: untick coming soon for WoW Forever in the admin Games screen and assign the tag `wow-forever`.

9 Setup, build and deployment

9.1 Requirements

Tool	Version
Java (JDK)	25
Maven	3.9
Node.js and npm	A version supported by Angular 21
Services	A Supabase project (PostgreSQL, Auth, Storage), Render and Vercel accounts

9.2 Configuration

The backend reads three layers; each one overrides the previous:

1. `backend/app/src/main/resources/application.yml`: versioned, no secrets. Gzip, connection pool, Swagger off.
2. `application.properties` in the same folder: local only, ignored by git. Secrets and local values.
3. Environment variables (Render). Spring maps `APP_AUTH_COOKIE_SECURE` to `app.auth.cookie-secure`, and so on.

Setting	Purpose
`spring.datasource.url`, `.username`, `.password`	PostgreSQL connection (Supabase pooler)
`supabase.url`, `supabase.anon-key`, `supabase.service-role-key`	Supabase Auth and Storage
`supabase.jwt.secret` / `.algorithm` / `.audience`	Token verification: empty secret uses the JWKS of `supabase.url` (ES256); audience `authenticated` by default
`app.frontend.url`, `app.frontend.set-password-path`	Where the email links point
`app.cors.allowed-origins`	Origins allowed with credentials (production: the Vercel domain)
`app.auth.cookie-secure`, `.cookie-same-site`, `.refresh-cookie-max-age-seconds`	Refresh cookie; `secure` must be true in production
`springdoc.api-docs.enabled`, `springdoc.swagger-ui.enabled`	Off by default; keep them off in production
`logging.level.*`, `logging.pattern.console`	Console logging with the request id

> **NEVER COMMIT SECRETS** Keys and database credentials go only in the ignored local file or in Render. The repository is public.

9.3 Build and run

```
cd backend
mvn clean install                 # every module, with the tests
java -jar app/target/ntg.jar      # API on port 8080

cd frontend
npm install
npm start                         # http://localhost:4200, /api forwarded to :8080
npm run build                     # production build
```

From an IDE, run `com.adrianpmdev.Application` in the `app` module. `.mvn/jvm.config` lets Lombok compile on Java 25 without warnings.

> **IDE OUT OF STEP** After the module structure changes (modules added, packages moved) the Java support of an IDE can leave broken classes in `target/classes`, which shows as `NoClassDefFoundError` of a bare class name at start-up. Run `mvn clean install` and, in VS Code, "Java: Clean Java Language Server Workspace".

9.4 Deployment

Part	Platform	How
Backend	Render	Multi-stage `Dockerfile`: Maven + JDK 25 builds the `app` module; a JRE 25 image runs `ntg.jar` as an unprivileged user, with the memory flags for 512 MB
Frontend	Vercel	Static build; `vercel.json` rewrites `/api/*` to Render, falls back to `index.html`, sets cache and security headers
Database, Auth, Storage	Supabase	Managed project; SQL files run by hand; credentials as environment variables of Render
Keep-alive	External cron	`GET /keep-alive` every 10 minutes keeps the free Render instance awake (about 730 of its 750 monthly hours)

- Both platforms deploy from `main` and only when their own folder changes. Vercel's Ignored Build Step is `[ "$VERCEL_GIT_COMMIT_REF" != "main" ] || git diff HEAD^ HEAD --quiet -- .`, so branches do not create previews.
- The Dockerfile copies each module's `pom.xml` and `src` explicitly: a new game module needs two `COPY` lines.
- Work on a branch per feature and merge into `main` when it is tested; only `main` stays between features.

> **WHAT NEEDS A DEPLOY** Data files of a module (`data.json`, `config.json`, `types.json`) reach production with a backend deploy. Pictures and the catalog change from the admin screens without any deploy. Structural SQL changes must run in Supabase before the code that needs them is deployed.

9.5 Troubleshooting

Symptom	Likely cause
401 on every call after login	Token not sent or expired; check the refresh cookie and `app.auth.cookie-*`
401 although the token is valid	Its audience is not `authenticated`, or the issuer does not match `supabase.url`
403 on a game route	The user lacks the tag, or the game is still coming soon
429 on login	Too many attempts; wait for `Retry-After`
Cookie not stored in production	`cookie-secure` false over HTTPS, or the call bypasses the Vercel rewrite
A screen breaks with "Content Security Policy" errors in the console	An inline script or a resource from a host the policy does not allow; adjust `vercel.json`
A picture upload answers 413 or 415	Over 4 MB, over 40 megapixels, or not JPEG/PNG
Pictures do not show, or uploads answer 502	A bucket is missing, the service role key is wrong, or Storage is unreachable
Invitation link opens the wrong site	`app.frontend.url` points to the wrong address
WoW data or Pokémon types answer 503	A data file of the module is missing or invalid

10 Adding a new game

A game is one backend module, one frontend folder, one catalog entry and, if it needs tables, one SQL file. Nothing in the platform changes. Below, `<game>` is the lowercase id (for example `chess`): it is the module name, API path, tag key, catalog key, URL and frontend folder.

10.1 Integration points at a glance

Layer	File or place	What to add
Backend	`backend/games/<game>/`	Module POM, code, `GameModule` bean, optional SQL file
Backend	`backend/pom.xml`	`<module>games/<game></module>` and the version in dependencyManagement
Backend	`backend/app/pom.xml`	Dependency on the game
Backend	`backend/Dockerfile`	Two `COPY` lines
Database	Supabase	Run the game's SQL file, if any
Frontend	`frontend/src/app/games/<game>/`	Layout, routes, views, API service, `<game>.game.ts`
Frontend	`games/games.ts`	The game's `GameModule` in `GAME_MODULES`
Admin screens	Games and Pictures	Catalog entry (creates the tag), cover, release

10.2 Backend

Step 1 · Create the module

Copy `backend/games/pokemon/pom.xml` to `backend/games/<game>/pom.xml` and change the `artifactId`. It depends on `platform-api`, Spring Web, Spring Data JPA (only if it has tables) and springdoc; never on `core` or Spring Security. Add the module to `backend/pom.xml` (modules and dependencyManagement) and as a dependency of `backend/app/pom.xml`.

Step 2 · Write the code

Under `com.adrianpmdev.<game>.features.<feature>`, with the feature layout. Entities use `@Table(name = "...", schema = "<game>")` and controllers the base path `/api/v1/<game>`. Static data can be a JSON file in `src/main/resources/<game>/` served with `JsonResponses`.

Step 3 · Declare the game

```
@Configuration
public class ChessModule {
    @Bean
    GameModule chessGameModule() {
        return GameModule.of("chess", "/api/v1/chess");
    }
}
```

Use `new GameModule(tag, path, List.of(new PlayerWrite(...)))` to open write routes to players (the game must then check the caller with `CurrentUser`). Implement `ImageSlotProvider` if the game needs public pictures.

Step 4 · Dockerfile and build

```
COPY games/chess/pom.xml games/chess/pom.xml
COPY games/chess/src games/chess/src
```

Then run `mvn clean install`.

10.3 Database

If the game has tables, write `backend/games/<game>/sql/<game>.sql` following the conventions of chapter 5 (schema `<game>`, row level security on every table, a restrictive policy for any bucket) and run it in Supabase.

10.4 Frontend

1. Create `frontend/src/app/games/<game>/` with its layout, theme (`styles/`), views, models and API service on `${API_BASE_URL}/<game>`.
2. Write `<game>.routes.ts` (player routes) and `<game>-admin.routes.ts` if it needs management screens.
3. Export the game from `<game>.game.ts` and list it in `games/games.ts`.

```
export const CHESS_GAME: GameModule = {
    id: 'chess',
    routes: () => import('./chess.routes').then(m => m.CHESS_ROUTES),
    adminRoutes: () => import('./chess-admin.routes').then(m =>
m.CHESS_ADMIN_ROUTES),
    adminItems: [{ label: 'GESTIÓN', route: '/admin/chess/...', icon: '▣' }]
};
```

10.5 Release

1. In the admin Games screen, add the game with its key (the tag is created too). It starts as coming soon: admins can open it, players cannot.
2. Upload its cover from the same screen, and any other picture it declares in the Pictures screen.
3. Assign the tag to the users who should play it.
4. When it is ready, untick coming soon. No deploy is needed.

10.6 Definition of done

- The module depends only on `platform-api` and builds with `mvn clean install`.
- Its `GameModule` is declared, and the Dockerfile copies the module.
- Its SQL file (if any) is in the module and has been run in Supabase, with row level security.
- The frontend folder, `<game>.game.ts` and the entry in `games/games.ts` exist, and no other game is imported.
- Every screen works at 320, 360, 768 and 1280 px.
- A user without the tag gets 403 on the API and cannot open the route; a tagged user can read; only an admin can write.
- The game has its own chapter in this manual, a row in Game modules at a glance and a line in the changelog.

11 Appendix

11.1 Conventions

Area	Convention
Language	Code, comments and documentation in English; the user interface in Spanish
Headers	Every source file starts with a short English header comment and the @author tag
Java packages	`com.adrianpmdev.<module>.features.<feature>.<layer>`; shared contracts in `com.adrianpmdev.platform`
API paths	`/api/v1/<game>/<plural-resource>`, kebab-case
Database	snake_case, one schema per module, row level security everywhere, DDL next to the module
Frontend	Hyphen-separated names, selector prefix `app-`; a game never imports another game
Identifiers	One game id everywhere; schema and Java package without the hyphen
Line endings	CRLF in every text file, enforced by `.gitattributes`
Git	One branch per feature, merged into `main` when tested
Secrets	Never committed

11.2 Glossary

Term	Meaning
Tag	Key assigned to a user that unlocks a game; becomes the authority `TAG_<KEY>`
Catalog	The list of games in `core.game`, served by `GET /games` to each user
Coming soon (upcoming)	Catalog state of a game that is listed for everyone but only admins can open
GameModule	How a game declares itself: tag and API path in the backend; id and routes in the frontend
platform-api	The module of contracts shared by core and the games
Port	A contract a game uses for a platform service (current user, storage, compression)
Picture slot	A public picture a module declares, with its key, group and label
Access / refresh token	Short-lived JWT kept in memory / long-lived token in an HttpOnly cookie, rotated on use
Problem details	RFC 7807 error format used by every API error

Term	Meaning
Request id	Value of `X-Request-Id`, present in every log line and error response
Randomlocke	A Pokémon run with shared rules, a fixed roster and several tournaments
Series / bye	A best-of-3 match / a bracket series against an empty side
Points adjustment	Points added to a player's series wins in one tournament
Slug	Lowercase key of a class or race made from its name; used in URLs
Build code	Compact text of a talent build used in share links

11.3 Known gaps and ideas

11.3.1 Platform

- The admin dashboard shows placeholder activity and system figures; it should show real data.
- There is no screen for a user to edit their own profile, although `PUT /users/me` exists.
- Automated tests cover the picture compressor, the authority cache, the sign-in limits, the token checks and the keep-alive; the account flows, guards and interceptor are tested by hand.
- Schema changes are SQL files run by hand.

11.3.2 Pokémon

- A player can only upload their team after an admin links their account.
- The grand final is a single best-of-3 without bracket reset; imported six-player brackets do not advance on their own.
- The artwork of each Pokémon game is still a URL typed by hand, not an uploaded picture.
- In the imported data, one series of "Torneo 2 - LvL 43" stops at 1-0 and two games of HeartGold's final tournament reuse a replay of another tournament.

11.3.3 WoW Forever

- Updating the game data is manual (download or edit, replace, redeploy).
- Three trees are one build behind wow-forever.gg, so their share codes may differ from that site.
- Icons come from a third-party host with no local copy; the origin and licence of the tree and launcher pictures are not recorded.
- Ticked Legacy challenges live in each browser; Dungeons y loot is not built; the launcher news are placeholders.

11.4 Changelog

Version	Date	Change
2.0	2026-10-27	Platform refactoring. Backend modules `platform-api`, `core`, `games/*` and `app`; games declare themselves (`GameModule`). Game catalog and public pictures with server-side compression, managed from the admin screens; tags created with each game; coming-soon enforced by the backend. WoW Forever data served as ready models (`data.json` + `config.json`), its settings removed; Pokémon type chart served by the backend. Free-tier techniques (caches, gzip, ETag, daily ping, JVM flags, browser cache). Security: sign-in limits, token audience, CSP and headers, row level security everywhere, non-root image. Responsive review of every screen. Manual rewritten: one chapter per game, no repeated content
1.4	2026-10-27	Pokémon module rebuilt: nine-table schema, double-elimination bracket, computed standings, team pictures in a private bucket, old data migrated
1.3	2026-10-26	WoW Forever game data centralized in `data.json`; Spellbook, Legacy System and Razas y racials added
1.2	2026-10-26	Refactoring outside the Pokémon module: shared versions, shared helpers, CRLF line endings
1.1	2026-10-25	WoW Forever module added; document restructured
1.0	2026-10-22	First edition